8. Анализ на резултатите

8.1. Сравнение на ускорението (едри задачи)

При нехомогенно натоварване статичното разпределение е неефективно - процес 0 е претоварен.

- При p=4: Sp = 1.34 (очаквано ~4.0)
- При p=8: Sp = 1.15 (дори по-лошо от p=4!)

P2P Full Graph показва най-добри резултати:

- При p=4: Sp = 1.81 (+35% спрямо статично)
- При p=8: Sp = 1.89 (+64% спрямо статично)

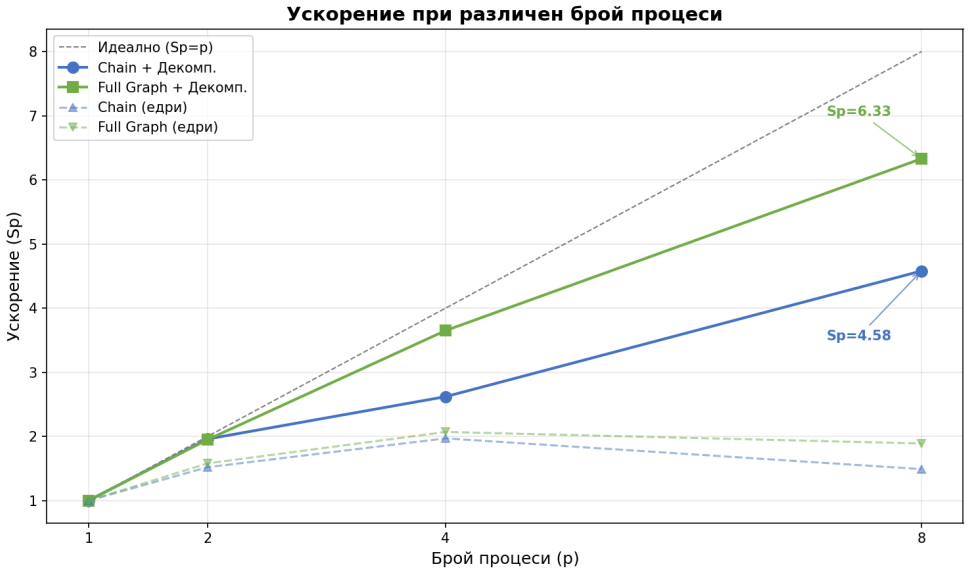
8.2. Влияние на декомпозицията (Сценарий 3)

Декомпозицията драматично подобрява ускорението:

Топология	Без декомп. (Sp)	С декомп. (Sp)	Подобрение
Chain	1.49	**4.58**	+207%
Full Graph	1.89	**6.33**	+235%

Ключови наблюдения:

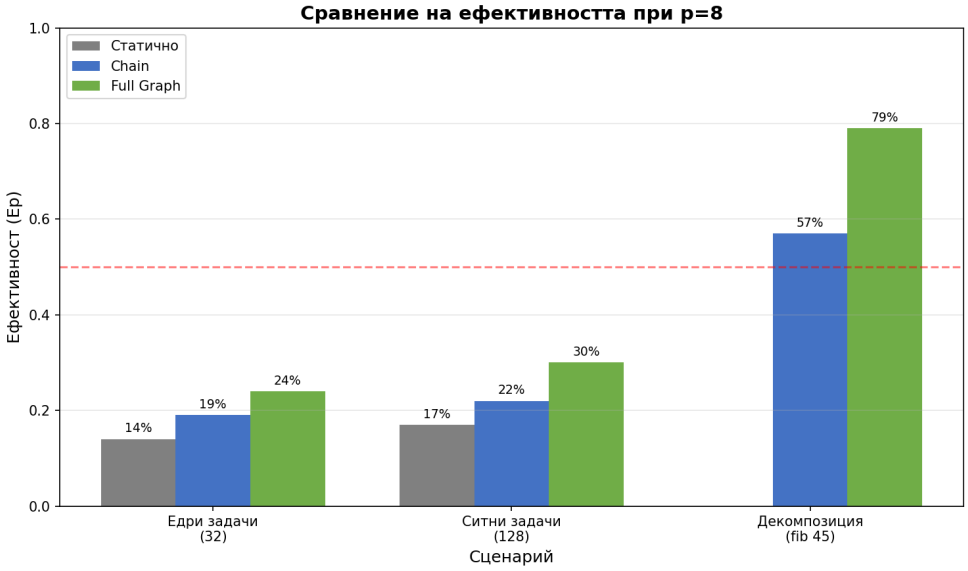
- Full Graph с декомпозиция постига **Sp=6.33** при 8 процеса (79% ефективност)
- Chain с декомпозиция постига **Sp=4.58** при 8 процеса (57% ефективност)
- Full Graph е **38% по-бърз** от Chain при декомпозиция (0.312s vs 0.426s)



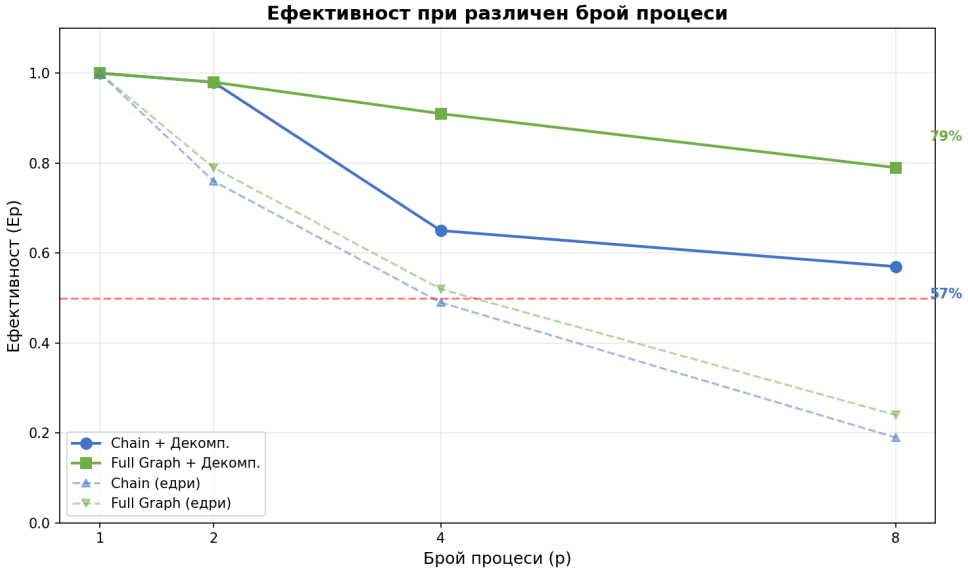
Фигура 1: Ускорение при различен брой процеси - декомпозицията драматично подобрява резултатите

8.3. Анализ на ефективността

Ефективността (Ep = Sp/p) показва колко добре се използват процесите:



Фигура 2: Ефективност при p=8 - декомпозицията постига 79% при Full Graph



Фигура 3: Ефективност при различен брой процеси

Ключови изводи от графиките:

- Без декомпозиция ефективността пада драматично при p=8 (14-30%)
- С декомпозиция Full Graph запазва **79% ефективност** при p=8
- Chain с декомпозиция достига **57% ефективност** при p=8
- Декомпозицията позволява истински паралелизъм - работата се разделя, не само задачите

8.4. Избор на оптимален праг

Прагът определя кога задачата се изчислява директно вместо да се декомпозира:

Топология	Оптимален праг	Тествани прагове	Ограничение
Full Graph	**30**	20-45	Няма - всички стабилни
Chain	**30**	20-45	Праг <30 блокира при p=1

Защо Chain блокира при нисък праг? При p=1 и праг<30 се генерират твърде много подзадачи, което препълва локалната опашка. Full Graph няма този проблем поради директната комуникация между всички процеси.

8.5. Сравнение на топологиите

Верижна топология (Chain):

- Всеки процес комуникира само с 2 съседа (ляв и десен)
- По-малко входящи заявки за обработка - добре за голям брой процеси
- **Недостатък:** Претоварен процес може да бъде достигнат само от съседите
- Подходяща за системи с локална (NUMA) топология

Пълен граф (Full Graph):

- Всеки процес може да комуникира директно с всеки друг
- **Предимство:** Директен достъп до претоварени процеси
- По-голям комуникационен overhead при много процеси
- Идеален за нехомогенно натоварване и малки клъстери

Ключов извод: При декомпозиция Full Graph е с **38% по-бърз** от Chain. Разликата идва от способността на всеки процес да краде работа директно от най-натоварения.

8.6. Обобщение на резултатите при p=8

Сценарий	Статично Sp	Chain Sp	Full Sp	Най-добър
Едри задачи (32)	1.15	1.49	1.89	Full (+64%)
Ситни задачи (128)	1.38	1.79	2.42	Full (+75%)
Декомпозиция fib(45)	-	4.58	**6.33**	Full (+38%)